



AutoCheck

Guion de presentación y defensa · 9 a 10 minutos

Autor:	Pedro Monesterolo
Curso:	Segundo Año
Carrera:	Técnico Superior en Desarrollo de Software
Docente:	Ing. Cristian Caón
Materias:	Completar con las materias correspondientes
Año:	2026

Cómo usar este guion

- El texto normal SE DICE en voz alta, con tus palabras (no lo memorices letra por letra).
- Las cajas ámbar son acotaciones: qué mostrar, cuándo clickear y cuánto tardar. No se leen.
- Tiempos totales: presentación ~6:00 + demo en vivo ~3:00 + cierre 0:30 = 9:30. Ensayalo con cronómetro dos veces.
- Regla de oro: la presentación es apoyo visual, no teleprompter. Mirá al jurado, no a la pantalla.

Mapa de tiempos. 0:00 – 9:30

#ñ undefined

Bloque	Contenido	Tiempo
1	Presentación + nombre de la app	0:00 – 0:40
2	Problema, usuarios, objetivo y alcance	0:40 – 2:00
3	Funciones principales	2:00 – 3:30
4	Base de datos: diseño y funcionamiento	3:30 – 4:40
5	Tecnologías + IA en el desarrollo	4:40 – 5:40
6	Problemas, soluciones y resultado	5:40 – 6:30

Bloque	Contenido	Tiempo
7	Demo en vivo (recorrido real)	6:30 – 9:00
8	Cierre	9:00 – 9:30

Bloque 1. Presentación y nombre · 0:00 – 0:40

#ñ undefined

DIAPOSITIVA: portada con logo + nombre. Parado, sonrisa, pausa de 2 segundos antes de arrancar.

Buenas tardes. Mi nombre es Pedro Monesterolo, de Segundo Año de la Tecnicatura en Desarrollo de Software, y les voy a presentar AutoCheck.

AutoCheck es una aplicación web para gestionar el mantenimiento de vehículos y flotas: services, gastos, documentos y vencimientos, todo en un solo lugar, con alertas que avisan antes de que algo venza.

Bloque 2. Problema, usuarios, objetivo y alcance · 0:40 – 2:00

#ñ undefined

DIAPOSITIVA: “El problema” con 3 datos concretos. No leas la lista: contala como una historia.

El problema es simple y lo tiene casi todo el mundo: nadie sabe con certeza cuándo le toca el service al auto. El aceite, los filtros, la VTV, el seguro, la patente... todo vive en cuadernos, en la cabeza o en planillas sueltas. El resultado son services olvidados que terminan en arreglos caros, multas por vencimientos y, si querés vender el auto, cero historial para mostrar.

Usuarios

DIAPOSITIVA: 4 perfiles con ejemplo de uno. Una frase por perfil.

La app está pensada para cuatro perfiles. El particular con su auto o su moto. La familia con varios vehículos compartidos. Las pymes con flota —logística, reparto, construcción, agro— que necesitan controlar cada unidad por patente o número interno y saber cuánto cuesta cada una. Y los talleres, que con el rol de administrador ven el parque completo de sus clientes.

Objetivo y alcance

El objetivo fue construir un sistema funcional que centralice historial, costos y vencimientos, y que prevenga en lugar de lamentar: el sistema compara fechas y kilometrajes reales contra intervalos configurables y avisa con anticipación.

El alcance incluye todo el ciclo: vehículos, mantenimientos, gastos, combustible, documentos, recordatorios, calendario, notificaciones, asistente, perfil y panel de administración. Quedó fuera a propósito —y lo tengo como trabajo futuro— lo multi-usuario por empresa y los avisos por email o WhatsApp.

Bloque 3. Funciones principales · 2:00 – 3:30

#ñ undefined

DIAPOSITIVA: 6 bloques con nombre + UNA frase cada uno. Nombralos mirando al jurado, la demo los muestra después.

Las funciones principales son seis. Primero, gestión de vehículos de cualquier tipo: hay un catálogo de 46 tipos, desde autos y motos hasta camiones y maquinaria, con patente opcional o número de unidad para flotas.

Segundo, historial de mantenimientos con estado automático: verde al día, amarillo próximo, rojo vencido, según kilómetros y fecha. Tercero, gastos con gráficos por categoría, por vehículo y evolución de seis meses, exportables a Excel y PDF.

Cuarto, control de combustible: cada carga lleva litros y kilómetros, y el sistema calcula el consumo real en km por litro, con alerta si cae más de un 15 por ciento. Quinto, documentos con vencimiento, recordatorios por fecha o kilometraje, calendario mensual y bandeja de notificaciones.

Y sexto, dos diferenciales: el dashboard calcula el costo por kilómetro y un puntaje de salud de 0 a 100 por vehículo, y se puede descargar la ficha completa en PDF, que es el libro de service digital: eso hasta suma valor de reventa.

Bloque 4. Base de datos · 3:30 – 4:40

#ñ undefined

DIAPOSITIVA: diagrama simple de las 10 tablas con flechas. Mostrá el SQL Editor de Supabase de fondo si querés.

La base es PostgreSQL en Supabase, con diez tablas. Profiles, atado uno a uno con el usuario de autenticación, con rol USER o ADMIN. Vehicles, que pertenece a un usuario y referencia al catálogo de tipos. Después, cinco tablas hijas del vehículo: mantenimientos, gastos, cargas de combustible, documentos y recordatorios. Más notificaciones por usuario y los dos catálogos compartidos: tipos de vehículo y tipos de mantenimiento con sus intervalos.

Tres decisiones importantes. Uno: no hay backend propio; toda la seguridad vive en la base con Row Level Security: cada usuario solo ve lo suyo y el admin ve todo, usando funciones como owns_vehicle e is_admin para no generar recursión en las políticas. Dos: un trigger crea el perfil automáticamente al registrarse. Tres: los archivos van a Storage, con imágenes públicas y documentos privados descargados por URL firmada.

Si te preguntan “¿por qué Supabase y no tu propio backend?”, respondé: autenticación, base relacional, archivos y seguridad por fila en un solo servicio gratuito, ideal para un MVP sin servidor que mantener.

Bloque 5. Tecnologías e IA · 4:40 – 5:40

#ñ undefined

DIAPOSITIVA: logos o tabla de 6 filas. Decí versiones exactas: demuestra dominio.

Del lado del cliente, Angular 20 con componentes standalone y signals para el estado reactivo, formularios reactivos con validadores propios, Tailwind CSS versión 4 con modo oscuro por clase, Chart.js para gráficos, y jsPDF con autotable más SheetJS para exportar PDF y Excel. Del lado del servidor, Supabase:

autenticación por email, PostgreSQL 15, Storage y RLS. Node 22 para desarrollar, y deploy automático a GitHub Pages con cada push a main.

Uso de IA durante el desarrollo

OJO: acá preguntan “IA durante el desarrollo”, no el chatbot de la app. Separá las dos cosas en una frase.

Usé inteligencia artificial como asistente de programación durante todo el proyecto: para diseñar el esquema de datos, generar los seeds de prueba, depurar errores de compilación y RLS, y redactar documentación. Siempre revisando y entendiendo cada cambio antes de aceptarlo: la IA proponía, yo decidía. Y aclaro la diferencia: además, la app incluye su propio asistente, AutoCheck IA, que responde usando los datos reales del usuario con un motor local de reglas, sin inventar nada.

Bloque 6. Problemas, soluciones y resultado · 5:40 – 6:30

#ñ undefined

DIPOSITIVA: tabla problema! solución, 4 filas. Contalos como batallas ganadas concretas.

Cuatro problemas que me enseñaron mucho. Primero, al migrar el diseño a Tailwind, las clases viejas pisaban a las nuevas y la app se veía rota: lo resolví pasando el estilo global a CSS nativo de Tailwind 4 con tokens propios y una capa de compatibilidad, y vaciando los SCSS viejos que chocaban.

Segundo, el proyecto pedía Node 22 y tenía instalado Node 20, que Angular 20 rechaza: lo detecté por el error del CLI y cambié de versión con npm. Tercero, en Supabase choqué con el error 404 PGRST205 y columnas inexistentes: era haber corrido los scripts en otro proyecto; la solución fue igualar el ref del proyecto y recargar el caché del esquema, más políticas con funciones security definer para evitar recursión en RLS.

Y cuarto, en los formularios los inputs saltaban cada vez que aparecía un error de validación: reservé el espacio del mensaje con CSS y posicioné el error en absoluto, así nada se mueve. Como resultado: una app completa, responsive, con modo oscuro, catorce rutas, diez tablas y datos demo que muestran todos los estados. Funciona, se entiende y se puede probar en vivo, que es lo que van a ver ahora.

Bloque 7. Demo en vivo · 6:30 – 9:00

#ñ undefined

CERRÁ la presentación y abrí la app ya logueada como flota@test.com (contraseña AutoCheck2024!). Tené Supabase abierto en otra pestaña por si preguntan por la base. Hablá mientras clickeás, sin apuro pero sin pausas largas.

“Empiezo logueado como una empresa de logística con cinco unidades. Este es el dashboard: de un vistazo veo una Sprinter vencida, el timeline de 90 días, el costo por kilómetro y la salud de cada unidad.” (30 segundos)

“Entro a la Sprinter: el aceite está vencido por kilómetros y por fecha. Voy a Documentación del Iveco: el seguro vencido hace 20 días, en rojo. Esto es exactamente lo que evita una multa.” (30 segundos)

“Ahora cargo un service en la Hilux: elijo el tipo, fecha y kilómetros, y fíjense que el sistema ya me calcula el próximo y me ofrece crear el recordatorio. Guardo... y el recordatorio existe. Cierro el loop preventivo.”

(45 segundos)

“En Combustible, la Hilux muestra la alerta: el último consumo cayó más de 15 por ciento. En Calendario veo el mes con los vencimientos. Y en la ficha del vehículo descargo el libro de service en PDF.” (40 segundos)

“Termino en el panel de administración: estadísticas globales, usuarios con cambio de rol y los catálogos que alimentan toda la app. Y en Supabase se ve la fila recién creada: interfaz, lógica y base conectadas.” (35 segundos)

CHECKLIST PREVIO (hacerlo la noche anterior, no el día): app abre, login flota/familia/admin funciona, seed corrido (flota con alertas visibles), exportar PDF descarga, Supabase abre, video/imágenes ok si usás apoyo, copia de respaldo del proyecto. Si algo falla en vivo: “esto muestra el manejo de errores” y seguís al siguiente punto. Nunca te quedés trabado.

Bloque 8. Cierre · 9:00 – 9:30

#ñ undefined

Para cerrar: aprendí a llevar un producto real de punta a punta —modelo de datos con seguridad a nivel de fila, frontend reactivo con estado en signals, y a depurar problemas reales de versiones, permisos y diseño. A futuro: flota compartida multi-usuario y avisos por email y WhatsApp. Y el aporte principal: convertir el cuaderno y la memoria en datos que avisan solos. Gracias, quedo atento a sus preguntas.

Anexo A. Preguntas anticipadas y respuestas

#ñ undefined

Estudiá estas 10. Si no sabés algo: “buena pregunta, no lo tengo verificado; lo implementé así por X y lo revisaría por Y”. Nunca inventes.

- ¿Por qué Angular y no React? Standalone + signals dan reactividad sin boilerplate; CLI con build escases y lazy loading por ruta gratis.
- ¿Qué es un signal y dónde lo usás? Estado reactivo granular: vehicles(), page(), theme(); los computed derivan paginados, timeline y costos sin stores externos.
- ¿Cómo evitás que un usuario vea datos ajenos? RLS: owns_vehicle/is_admin; el JWT solo lleva anon key pública.
- ¿Qué pasa si borro un vehículo? Cascada: se borran sus services, gastos, docs, recordatorios, combustible y notificaciones. El borrado pide confirmación.
- ¿Cómo calculás el próximo service? recommended_km/months del tipo + último registro; fecha con clamp a fin de mes; no duplica recordatorios pendientes iguales.
- ¿Y si el vehículo no tiene patente? unit_number + fallbacks S/D; la patente es opcional y va en mayúsculas.
- ¿El costo/km con qué datos? TCO (gastos + services) dividido por km actual menos km del primer service; sin base muestra “—”.
- ¿La IA inventa respuestas? No: motor local de reglas sobre datos reales; sin datos, dice qué cargar.
- ¿Qué mejorarías con más tiempo? Organizaciones multi-usuario, emails/WhatsApp, PWA con push, OCR de facturas.
- ¿Qué hace el seed? Borra la demo y regenera 4 usuarios, 8 vehículos y movimientos con fechas relativas a hoy; validado en Postgres real.

Anexo B. Adaptación al video de 5 min (Puertas Abiertas)

#ñ undefined

Mismo material, otro ritmo: 0:20 presentación + 0:30 problema + 0:40 solución + 0:30 contexto real (un taller, un galpón, la calle) + 2:30 app + 0:30 cierre.

- Presentación (20s): nombre, carrera, “AutoCheck”.
- Problema (30s): el service olvidado, quién lo sufre, por qué lo elegiste.
- Solución (40s): qué hace, qué permite, quién la usaría.
- Contexto real (30s): mostrará dónde se usaría (cochera, taller, ruta).
- App (2:30): SOLO lo visual — login flota, dashboard con alertas, cargar un service con próximo automático, calendario y ficha PDF.
- Cierre (30s): qué aprendiste, qué mejorarías, aporte principal.
- Portada obligatoria primero: app, autor, curso, carrera, docente, materias, 2026.